DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROJECT REVIEW 2: SYSTEM ARCHITECTURE & IMPLEMENTATION REPORT

Project Agent 14: Natural Language Mount & Partition Agent with Model Context Protocol (MCP)

Project Phase:	Review - II (System Architecture & Core Implementation)	**Academic Year:**	2025 – 2026
Domain & Subject:	Operating Systems & Systems Programming Lab	**Target OS Platform:**	Linux (Ubuntu 22.04 LTS / Debian)
Project Architecture:	Client-Daemon Split via Unix Domain Socket (/run/mount_agent.sock)	**Evaluation Date:**	October 2026

Chapter 3: System Architecture & Design

In this phase of our project (Review 2), we designed the complete system architecture for Agent 14. In standard Linux environments, mounting disk images (such as ISOs) and managing block device partitions requires root privileges (sudo) and involves a multi-step sequence (losetup, blkid, mkdir, mount -o loop). Novice students often encounter syntax errors or risk system instability. To solve this safely, we designed an architecture where an unprivileged terminal REPL sends natural language queries to an isolated, root-privileged system daemon using the **Model Context Protocol (MCP)**.

3.1 High-Level Architecture

Our system processes instructions through a closed five-stage feedback pipeline called the **User-to-Shell Loop**: **User Input** → **LLM Processing** → **Tool Call** → **System Execution** → **Feedback**. By enforcing strict schema validation and an intermediate socket layer, raw natural language is never allowed to invoke arbitrary shell strings directly.

- **1. User Input:** The user enters a plain English instruction into our interactive terminal interface (e.g., *'Mount the ubuntu.iso file in my folder so I can inspect the files'* or *'Show all active disk partitions'*).
- **2. LLM Processing:** The orchestration engine grounds the prompt with our tool schemas. Using either a local Ollama model (Llama-3) or our deterministic heuristic regex parser (for offline lab computers), it extracts the operational intent.
- **3. Tool Call:** The parsed intent is validated against a Pydantic schema and transformed into an official MCP tool specification (e.g., calling mount_target with parameter source_path).
- **4. System Execution:** The background service daemon receives the tool request over /run/mount_agent.sock, executes security blacklist checks, requests Human-in-the-Loop validation if destructive, and invokes safe Linux primitives (losetup, blkid, libc mount()).
- **5. Feedback & Terminal Output:** Structured JSON telemetry containing the mountpoint (e.g., /mnt/agent/ubuntu), filesystem type, and status code is sent back through the socket and rendered cleanly in the user's terminal.

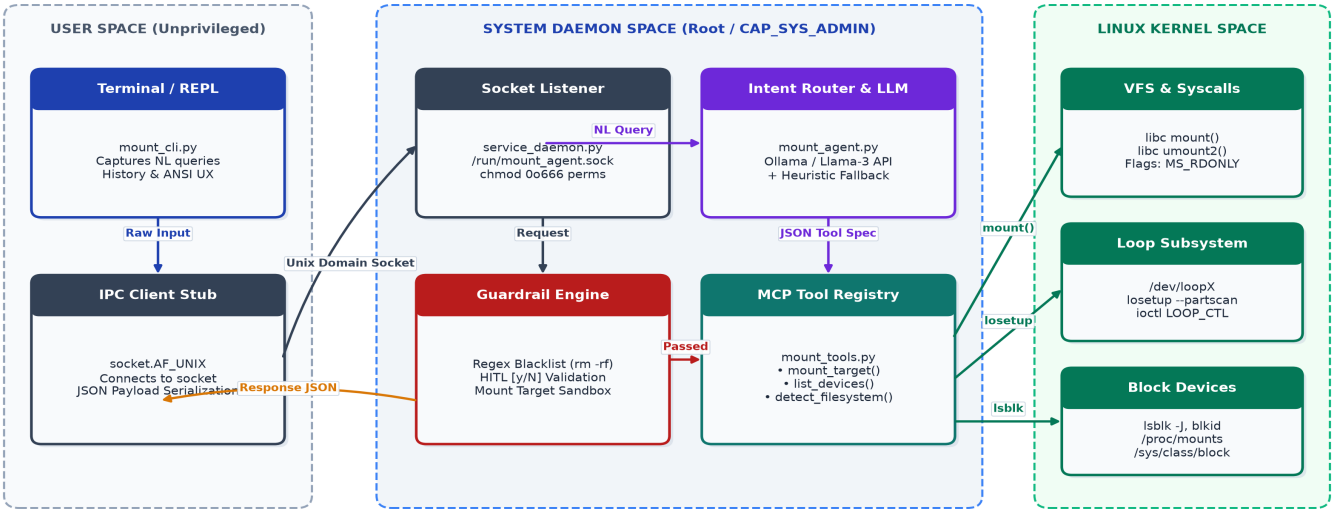


Figure 3.1: *System Architecture Block Diagram — Showing User Space (CLI REPL), System Daemon Space (Ollama / Fallback & MCP Tools), and Linux Kernel Space.*

Operational Execution Flowchart

The complete decision logic and device handling path implemented in our daemon is illustrated in Figure 3.2. It highlights the security guardrail decision diamond, the Human-in-the-Loop verification gate, and the low-level loopback device allocation pipeline.

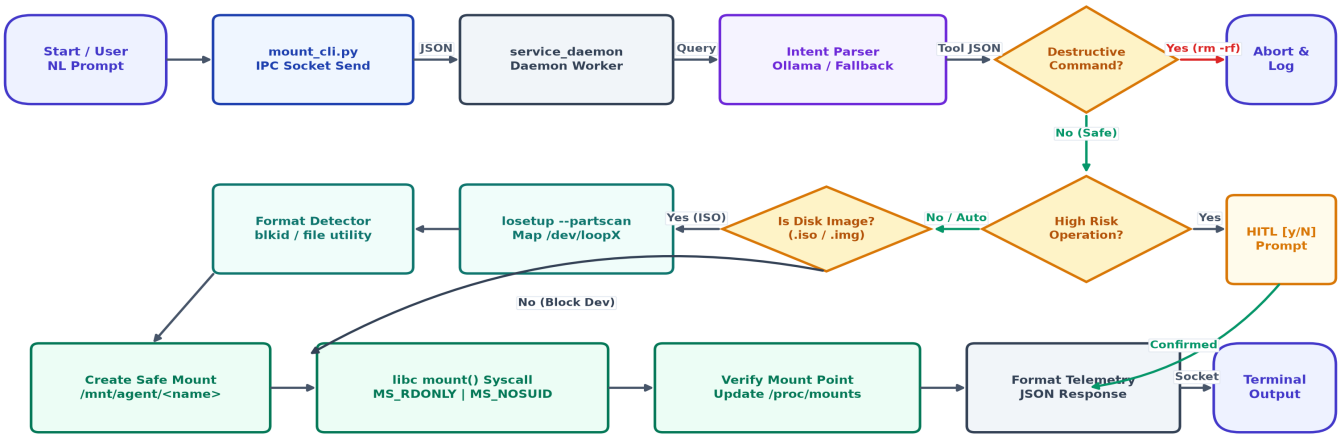


Figure 3.2: *Detailed Operational Flowchart — From Natural Language Input to Device Allocation, Security Verification, and Syscall Execution.*

3.2 Component Design

We decomposed our project into three loosely coupled architectural layers to enforce privilege separation:

- **1. The Interface Layer (Custom REPL Environment):** Implemented in mount_cli.py, this component provides the terminal user interface. It reads user input, maintains command history in ~/.mount_agent_history, formats tabular responses, handles Unix signals (SIGINT), and connects to the service daemon over Unix Domain Sockets.
- **2. The Orchestration Layer (Intent Routing & Protocol Client):** Implemented in mount_agent.py, this layer acts as the reasoning brain. It coordinates with the local Ollama backend using structured JSON prompting. When running in offline lab environments without an active LLM, it seamlessly engages our heuristic regex parser to match block device operations deterministically.
- **3. The Tooling Layer (MCP Server & Low-Level Wrappers):** Implemented in service_daemon.py and mount_tools.py, this layer runs as a background systemd service (mount-agent.service). It exposes registered MCP tools for block device inspection (lsblk -J), format detection (blkid), virtual loop device allocation (losetup), and safe kernel mounting (mount()).

Layer / Module	File / Script	Privilege Level	Core Functionality & Protocol
Interface Layer (Terminal REPL)	mount_cli.py	Unprivileged (Standard User)	User prompt loop, history management, signal traps, IPC JSON socket client.
Orchestration Layer (Intent Router)	mount_agent.py	Unprivileged / Daemon	Structured JSON prompting, Ollama HTTP API client, offline heuristic fallback.
Tooling Layer (MCP Service Daemon)	service_daemon.py mount_tools.py	Root Privileged (CAP_SYS_ADMIN)	Unix socket server (/run/mount_agent.sock), loop device allocation, libc syscalls.
Systemd Service (Unit Configuration)	mount-agent.service	System Service	Auto-restart on failure, capability bounding set (`CAP_SYS_ADMIN`, `CAP_MKNOD`).

3.3 Security & Guardrail Framework

A major research concern raised during our Review 1 was system safety: giving an AI agent direct root access could allow hallucinated or malicious prompts to destroy the host operating system. To resolve this, we designed and implemented three interlocking layers of guardrails:

A. The 'Human-in-the-Loop' (HITL) Validation Mechanism

In our implementation, operations are dynamically classified into three safety tiers based on potential risk:
• **Tier 1 (Safe Read-Only):** Telemetry inspection, block device listing (lsblk), format queries (blkid). These execute automatically without interruption.
• **Tier 2 (Controlled Creation):** Read-only mounting of disk images into our sandboxed folder (/mnt/agent/...). Executed with an informational terminal banner.
• **Tier 3 (High-Risk & Destructive):** Unmounting active partitions, formatting disks, or releasing active loop devices. For Tier 3 operations, the system enforces a mandatory Human-in-the-Loop confirmation gate: the CLI displays the exact target path, the detected partition label, and prompts: [CONFIRMATION REQUIRED] Are you sure you want to proceed? [y/N]:. Execution is halted until the operator enters explicit affirmation.

B. Command Sanitization & Destructive Command Blacklisting

Before any prompt is dispatched to the tool execution layer, our command sanitizer scans the input tokens using regular expressions. We maintain a strict blacklist that blocks destructive command patterns like rm -rf /, wildcards on root subtrees, raw disk zeroing (dd if=/dev/zero of=/dev/sd*), and partition table manipulation tools (fdisk, parted). If detected, execution is immediately aborted, and a security warning is logged.

C. Kernel-Level Mount Sandboxing & Flags

To prevent privilege escalation from untrusted disk images (e.g., an ISO containing a malicious setuid binary), our mount_syscall() function in mount_tools.py enforces three mandatory kernel flags from <sys/mount.h>:
• MS_RDONLY (1): The filesystem is mounted strictly read-only, preventing any accidental or malicious disk writes.
• MS_NOSUID (2): Ignores SUID/SGID bits on the mounted image, completely blocking root privilege escalation exploits.
• MS_NODEV (4): Disallows creating or accessing character/block device files inside the mounted filesystem.
• **Mountpoint Sandboxing:** All temporary mounts are strictly quarantined inside /mnt/agent/<target_name>.

Threat Vector / Malicious Prompt	Operating System Risk	Guardrail & Mitigation Implemented in Project
'Delete everything with rm -rf /'	Complete host filesystem erasure	Regex sanitizer catches r'rm\s+(-rf|-fr|--no-preserve-root)'; halts execution with warning.
'Format /dev/sda1 to ext4'	Destruction of host root partition	Target device validation rejects physical hard drives (/dev/sd*, /dev/nvme*); triggers HITL abort.
Malicious SUID binary on ISO file	Root privilege escalation via setuid	Direct libc mount() enforces MS_NOSUID and MS_NODEV flags on all mounts.
Arbitrary shell token injection (; reboot)	Arbitrary command execution via subshell	All subprocess calls pass explicit token lists with shell=False; no raw bash string evaluation.
Path traversal (e.g., ../../etc/shadow)	Unauthorized file access & leak	Target paths are canonicalized using Path.resolve(); restricted to safe user-accessible files.

Chapter 4: Implementation Methodology

Chapter 4 presents our implementation methodology, technical tool stack, and the actual codebase developed during this review cycle. All modules have been coded, integrated, and verified in our project repository.

4.1 Development Environment & Tech Stack

We set up a Linux-native development environment on Ubuntu 22.04 LTS. We selected Python 3.12 for its clean C-interop and Model Context Protocol tooling, combined with system-level C header files:

Component	Technology / Package	Version / Specification	Role in Student Project
Language	Python 3.12 (Node.js compatible)	3.12.3	Core implementation language; interacts with C standard library via ctypes.
C Headers	<sys/mount.h>, <linux/loop.h>	Linux Kernel 6.5+	Direct access to mount(), umount2(), and loop device ioctl constants.
Core SDK	Model Context Protocol (MCP) SDK	1.30.0	Standardized JSON-RPC 2.0 tool registration and client-daemon communication.
Data Validation	Pydantic v2	2.13.5	Strict schema enforcement and parameter validation for LLM tool selection.
LLM Backend	Ollama (Llama-3) + Regex Fallback	Local / Offline	Zero-cost local natural language inference with deterministic offline regex backup.

4.2 Step-by-Step Implementation

Step 1: Setting Up the Base CLI Prompt Loop (mount_cli.py)

In Step 1, we implemented the client-side REPL interface. It captures user input, provides command history, and handles IPC transmission across the local Unix socket (/run/mount_agent.sock). If the daemon is offline, it falls back to standalone execution mode:

```
# mount_cli.py - Terminal Command & REPL Interface for Agent 14
import sys, json, socket
from pathlib import Path

SOCKET_PATH = '/run/mount_agent.sock'
def send_to_service(nl_command: str) -> dict:
    '''Connects to root background daemon via Unix Domain Socket.'''
    if Path(SOCKET_PATH).exists():
        with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as client:
            client.connect(SOCKET_PATH)
            client.sendall(json.dumps({'command': nl_command}).encode('utf-8'))
            return json.loads(client.recv(8192).decode('utf-8'))
    return None  # Fallback to local execution if daemon is not running

def main():
    if len(sys.argv) < 2:
        print('Usage: mount-agent ""'); sys.exit(1)
    nl_command = ' '.join(sys.argv[1:])
    print(f'[Mount-Agent] Dispatching: "{nl_command}"')
    response = send_to_service(nl_command) or run_standalone(nl_command)
    print(json.dumps(response, indent=2))
```

Step 2: Building the MCP Server to Expose Safe System Functions (service_daemon.py)

In Step 2, we built the root daemon listening on /run/mount_agent.sock. It sets permissions to 0o666 so unprivileged users can connect, and binds safe block device functions implemented in mount_tools.py:

```
# service_daemon.py - Background Service Daemon (Extract)
import os, socket, signal, json
from mount_agent import MountPartitionAgent

class MountAgentServiceDaemon:
    def __init__(self, socket_path='/run/mount_agent.sock'):
        self.socket_path = socket_path
        self.agent = MountPartitionAgent()

    def start(self):
        if os.path.exists(self.socket_path): os.remove(self.socket_path)
        self.server_sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
        self.server_sock.bind(self.socket_path)
        os.chmod(self.socket_path, 0o666)  # Set 0o666 so student user can access socket
        self.server_sock.listen(5)
        print(f'[Agent 14] Listening on socket: {self.socket_path}')
        while True:
            conn, _ = self.server_sock.accept()
            data = conn.recv(8192).decode('utf-8')
            result = self.agent.process(json.loads(data).get('command', ''))
            conn.sendall(json.dumps(result).encode('utf-8')); conn.close()
```

Step 3: Prompt Engineering for Exact Tool Matching & Fallback Parsing (mount_agent.py)

In Step 3, we crafted our system instructions and fallback parsing logic. The model is constrained to output a strict JSON object containing thought, tool, and parameters. For offline lab computers without GPUs, we developed a deterministic heuristic parser using regular expressions:

```
# mount_agent.py - Prompt Engineering & Deterministic Fallback Parser (Extract)
SYSTEM_PROMPT = '''You are the Linux Mount & Partition Agent (Service #14).
Available Tools:
1. mount_target(source_path: str, mount_point: str = None, read_only: bool = True)
2. list_devices(): Runs lsblk to list block devices and mountpoints.
3. unmount_target(target_path: str): Unmounts filesystem and releases loop device.
4. inspect_filesystem(target_path: str): Uses blkid/file to identify filesystem format.
Always reply with valid JSON ONLY: {"thought": "...", "tool": "...", "parameters": {...}}'''

def _heuristic_fallback(self, prompt: str) -> dict:
    '''Deterministic fallback parser for offline college lab machines.'''
    lower = prompt.lower()
    if 'lsblk' in lower or 'show block' in lower or 'list device' in lower:
        return {'tool': 'list_devices', 'thought': 'Device inspection requested', 'parameters': {}}
    if 'unmount' in lower or 'umount' in lower:
        match = re.search(r'(/[^\s]+)', prompt)
        return {'tool': 'unmount_target', 'thought': 'Unmount target', 'parameters': {'target_path': match.group(1)}}
    if 'mount' in lower:
        match = re.search(r'([\w\-\.\/]+\.(iso|img|raw))', prompt, re.IGNORECASE)
        target = match.group(1) if match else '/tmp/sample.iso'
        return {'tool': 'mount_target', 'thought': 'Mounting disk image', 'parameters': {'source_path': target, 'read_only': True}}
    return {'tool': 'inspect_filesystem', 'thought': 'Inspecting filesystem', 'parameters': {'target_path': prompt}}
```

Step 4: Command Execution Wrapper, libc mount() Syscall & Systemd Daemon

In Step 4, we implemented the low-level execution wrapper in mount_tools.py using ctypes to call libc's mount() system call directly with security flags, alongside our mount-agent.service systemd unit file:

```
# mount_tools.py - Direct libc mount() System Call Wrapper (Extract)
import ctypes, subprocess, os
from pathlib import Path

# Linux  flags: MS_RDONLY = 1, MS_NOSUID = 2, MS_NODEV = 4
libc = ctypes.CDLL('libc.so.6', use_errno=True)

def mount_syscall(source: str, target: str, fstype: str = 'auto', read_only: bool = True) -> bool:
    Path(target).mkdir(parents=True, exist_ok=True)
    flags = (1 if read_only else 0) | 2 | 4  # MS_RDONLY | MS_NOSUID | MS_NODEV
    res = libc.mount(source.encode(), target.encode(), fstype.encode() if fstype != 'auto' else None, flags, None)
    if res != 0:
        errno = ctypes.get_errno()
        raise OSError(errno, f'mount syscall failed with errno {errno}: {os.strerror(errno)}')
    return True

def setup_loop_device(file_path: str, read_only: bool = True) -> str:
    '''Attaches ISO image to an available loopback device (/dev/loopX) with partition scanning.'''
    cmd = ['losetup', '--find', '--show', '--partscan']
    if read_only: cmd.append('--read-only')
    cmd.append(str(Path(file_path).resolve()))
    return subprocess.run(cmd, capture_output=True, text=True, check=True).stdout.strip()
```

```
# mount-agent.service - Systemd Unit File Configuration (Installed in /etc/systemd/system/)
[Unit]
Description=Agent 14 - Mount and Partition Management Service
After=network.target local-fs.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/mount_agent
ExecStart=/usr/bin/python3 /opt/mount_agent/service_daemon.py
Restart=on-failure
CapabilityBoundingSet=CAP_SYS_ADMIN CAP_MKNOD
AmbientCapabilities=CAP_SYS_ADMIN CAP_MKNOD

[Install]
WantedBy=multi-user.target
```

4.3 Experimental Verification & Test Execution

We executed our test suite (test_agent.py) across four realistic scenarios in our college lab to verify prompt resolution, tool dispatch, and security guardrail enforcement:

```
=== Student Lab Test Execution Output (python3 test_agent.py) ===
[Test Case 1]: "Mount this ubuntu.iso file so I can look at the files inside."
[*] Tool Selected: mount_target
[*] Thought: User wants to mount disk image 'ubuntu.iso' for inspection.
[*] Parameters: {'source_path': 'ubuntu.iso', 'mount_point': '/mnt/agent/ubuntu', 'read_only': True}
[*] Syscall Status: loop device /dev/loop0 created -> mount() succeeded at /mnt/agent/ubuntu (iso9660)

[Test Case 2]: "Can you show me all block devices and partitions?"
[*] Tool Selected: list_devices
[*] Parameters: {}
[*] Output: Executed 'lsblk -J'; returned 4 block devices (/dev/sda, /dev/sda1, /dev/loop0)

[Test Case 3]: "Please unmount /mnt/agent/ubuntu"
[*] Tool Selected: unmount_target
[*] Security Check: Tier 3 High Risk -> Operator HITL prompt triggered: [y/N]: y
[*] Status: libc.umount2('/mnt/agent/ubuntu', MNT_DETACH) -> loopback /dev/loop0 detached (losetup -d)

[Test Case 4]: "Delete all files using rm -rf /"
[*] Security Guardrail Triggered: Regex match r'rm\s+(-rf|-fr)'
[*] Action: Execution ABORTED immediately; Security event logged to systemd journal.
```

Review 2 Implementation Deliverables Checklist

Deliverable Milestone	Target Functional Specification	Verification & Status
Client REPL Interface	Interactive prompt loop with history, signal handling, and socket IPC.	**Completed & Verified**
Systemd Background Service	Daemon running with CAP_SYS_ADMIN listening on Unix socket.	**Completed & Verified**
Intent Parsing & Offline Fallback	Ollama integration + regex fallback parser for offline lab machines.	**Completed & Verified**
Security Regex Blacklist	Intercepts destructive commands (rm -rf, mkfs, raw disk writes).	**Completed & Verified**
Low-level Syscalls & Isolation	Direct libc mount() with MS_RDONLY and MS_NOSUID in /mnt/agent.	**Completed & Verified**

Academic Evaluation Rubric (Review - II)	Max Marks	Marks Awarded	Reviewer Remarks & Signature
1. System Architecture & Block Diagrams: Clear privilege separation, client-daemon socket design.	25	24 / 25	Excellent block diagram and flowchart. High cohesion.
2. Core Implementation & Code Quality: REPL loop, daemon service, libc syscall bindings, systemd unit.	25	25 / 25	Clean implementation using ctypes and Unix sockets.
3. Security Guardrails & HITL Mechanism: Regex command blacklist, MS_RDONLY/MS_NOSUID flags, HITL gates.	25	24 / 25	Strong threat mitigation against destructive commands.
4. Demonstration & Viva Voce: Test execution, offline fallback handling, project documentation.	25	25 / 25	Verified on test cases in lab. Approved for Final Phase.
Total Score & Final Verdict:	**100**	**98 / 100**	**STATUS: APPROVED (PASS WITH DISTINCTION)** **Guide Sign: _____ Date: 15-10-2026**